RootAccess

QA & Software Testing Engineer Curriculum

Comprehensive Course Syllabus, Tracks, Lesson Outcomes, Quizzes & Defect Labs

VERSION: 2026.1 • REVISION DOCUMENT FOR CURRICULUM UPDATES

1. Program Executive Overview

RootAccess is an industry-grade career learning platform designed to bridge the gap between textbook testing theory and real-world workplace execution. This curriculum document outlines the 5 core tracks, detailed lesson topics, practical checklists, certification quizzes, and workplace simulation labs present in the current release. Use this document to audit coverage, adjust lesson durations, add new technical tracks, or refine interview scenarios.

5 Active Tracks	~35 Est. Hours	9 In-Depth Lessons	4 Live Bug Scenarios
Career Tracks	Total Training	Core Lessons	Defect Simulator

2. Complete Track & Lesson Breakdown

Track 1: Manual QA & Defect Life Cycle Masterclass

Core Foundation Track • Estimated: 8 Hours

Master practical test case design, Equivalence Partitioning, Boundary Value Analysis, Jira defect reporting, and regression test suites.

Target Learning Outcomes:

- Write exhaustive, professional test cases using industry templates
- Apply Boundary Value Analysis (BVA) and Equivalence Partitioning (EP) to spot edge-case bugs
- Log clear, reproducible Jira bug reports with appropriate Severity and Priority levels
- Execute Smoke, Sanity, Regression, and Retesting verification cycles with confidence

Lesson Modules:

[BEGINNER • 25 Minutes] Lesson 1: Software Testing Life Cycle (STLC) & Modern QA Workflows

Summary: Understand the distinct phases of the STLC, from requirements analysis to test closure, and how QA fits into modern Agile/Scrum teams.

Key Detailed Topics:

- The 6 Core Phases: Requirement Analysis, Test

Planning, Test Design, Environment Setup, Execution, and Sign-off
- Shifting Testing Left: Involving QA during sprint grooming to prevent 60% of requirement ambiguities
- Evaluating Positive (happy path) vs Negative (error & edge) test scenarios
- Gherkin BDD syntax and test requirement decomposition

[INTERMEDIATE • 35 Minutes] Lesson 2: Test Design Techniques: Boundary Value Analysis (BVA) & Equivalence Partitioning (EP)

Summary: Learn the two most important mathematical test design techniques used by senior QA engineers to catch edge-case bugs with minimum test cases.

Key Detailed Topics:
- Equivalence Partitioning: Dividing input domains into valid and invalid classes to eliminate redundancy
- Boundary Value Analysis: Testing Min, Min-1, Min+1, Max, Max-1, Max+1 boundaries
- Why BVA is critical: Developers most frequently commit off-by-one errors (>= vs >)
- Real-world boundary bugs: Cart quantity validation, character limit overflows, password boundaries

[INTERMEDIATE • 30 Minutes] Lesson 3: Workplace Defect Logging: Jira Bug Report Standards

Summary: Learn how to write crisp, irreproachable bug reports in Jira that developers love. Master Severity vs Priority distinctions.

Key Detailed Topics:
- Standard Jira Fields: Title ([Module] What happens under condition), Environment, Preconditions, Steps to Reproduce (numbered), Expected vs Actual, Attachments
- Severity vs Priority: Technical Impact (Blocker/Critical/Major/Minor) vs Business Urgency (P0/P1/P2/P3)
- Defect Life Cycle states: Open, In Progress, Ready for QA, Retested & Verified, Closed, Deferred

' Manual QA & Defect Life Cycle Certification Quiz (4 Questions, 75% Passing)

&¡ Workplace Defect Simulation Lab: E-Commerce Checkout Defect Hunt (4 Real Bugs)

Track 2: SQL & Backend Database Audits for QA Engineers

Backend Integrity Track • Estimated: 6 Hours

Learn to verify backend data integrity, write multi-table JOINs, detect orphaned records, and validate database transactions during API and UI testing.

Target Learning Outcomes:

- Write production-grade SQL queries to verify frontend transactions against database tables
- Use INNER, LEFT, and RIGHT JOINs to detect missing or orphaned foreign key records
- Identify NULL edge cases, duplicate entries, and timestamp inconsistencies in staging databases
- Audit user balances, inventory decrements, and status state changes using aggregate functions

Lesson Modules:

[BEGINNER • 30 Minutes] Lesson 1: Verifying Frontend Transactions Against Relational Tables

Summary: Learn how to inspect the database after submitting a UI form or calling a payment endpoint to confirm true persistence.

Key Detailed Topics:

- Why UI success banners can deceive: asynchronous write failures and uncommitted transactions
- Writing SELECT queries with WHERE filters to audit orders, payment_status, and created_at timestamps
- Auditing inventory decrements and transactional line items in child tables

[INTERMEDIATE • 35 Minutes] Lesson 2: Multi-Table JOINs & Finding Orphaned Data Defects

Summary: Master SQL JOIN techniques to uncover referential integrity flaws and orphaned records in staging environments.

Key Detailed Topics:

- LEFT JOIN with WHERE parent.id IS NULL to isolate orphaned child records
- Auditing duplicate transactions and race conditions with GROUP BY and HAVING COUNT(*) > 1
- Data migration audits: ensuring foreign keys and data types remain intact

' SQL for QA Engineers Certification Quiz (3 Questions, 75% Passing)

&¡ SQL Database Verification Queries in Test Environments

Track 3: API Testing & Automation with Postman

API & Microservices Track • Estimated: 7 Hours

Learn REST API architectures, request parameterization, Bearer token authentication, status codes, and writing automated Chai assertion scripts in Postman.

Target Learning Outcomes:

- Understand HTTP methods (GET, POST, PUT, PATCH, DELETE) and HTTP status codes (2xx, 4xx, 5xx)
- Construct automated test scripts using Postman and Chai assertion library (pm.test, pm.expect)
- Extract dynamic response values (JWT tokens, resource IDs) and save them into environment variables
- Test edge cases: expired tokens, invalid payloads, SQL injection parameters, and rate limiting

Lesson Modules:

[BEGINNER • 25 Minutes] Lesson 1: HTTP Methods, Status Codes & Request Payloads

Summary: Deep dive into REST API architecture, request headers, query vs path parameters, and HTTP response codes.

Key Detailed Topics:

- Anatomy of an HTTP Request: Method, Endpoint URL, Headers, Query vs Path Params, Request Body
- Status Codes: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 422 Unprocessable, 500 Server Error
- Common API Anti-Pattern: Returning 200 OK with an error payload inside the body

[INTERMEDIATE • 35 Minutes] Lesson 2: Writing Automated Postman Assertions with Chai

Summary: Write test scripts in Postman to assert status codes, response time, JSON schema validation, and header security.

Key Detailed Topics:

- Using pm.test and pm.expect for status code, SLA latency (<500ms), and schema checks
- Chaining requests: Capturing JWT access_token from login and saving to pm.environment.set()
- Negative testing: Sending malformed JSON, empty bodies, and expired authentication headers

' API Testing & Postman Certification Quiz (3 Questions, 75% Passing)

&¡ Postman Collection Runner & Environment Token Chaining Lab

---

Track 4: Modern Web Automation with Playwright & TypeScript

Automation Engineering Track • Estimated: 9 Hours

Learn modern browser automation using Playwright. Master reliable locator strategies, auto-waiting, Page Object Models, and flaky test mitigation.

Target Learning Outcomes:

- Write robust end-to-end tests using Playwright Test Runner and TypeScript
- Use user-facing locators (getByRole, getByLabel, getByText) recommended by W3C standards
- Leverage Playwright auto-waiting to eliminate arbitrary sleep timers and flaky test failures
- Structure test suites with the Page Object Model (POM) pattern for high maintainability

Lesson Modules:

[INTERMEDIATE • 30 Minutes] Lesson 1: Robust Locators, Actions & Web-First Assertions
Summary: Learn why XPath and brittle CSS selectors cause test flakiness, and how to use Playwright accessibility-based locators.
Key Detailed Topics:
- Recommended locator hierarchy: getByRole, getByLabel, getByPlaceholder, getByText, getByTestId
- Why brittle selectors like xpath=//div[3]/form/button[2] break on redesigns
- Web-First Assertions: Automatic polling with expect(locator).toBeVisible() without hardcoded sleeps
- Writing an End-to-End User Login and Dashboard Verification test script in TypeScript

' Playwright Automation Certification Quiz (2 Questions, 75% Passing)
&¡ Playwright E2E Test Suite Execution against Local Staging App

---

Track 5: QA Interview Mastery & Career Blueprint

Career & Placement Track • Estimated: 5 Hours
Prepare for behavioral, scenario-based, and technical QA interviews with the STAR technique, bug triage negotiation, and salary benchmarking.

Target Learning Outcomes:
- Structure behavioral answers using the STAR method (Situation, Task, Action, Result)
- Handle classic interview traps: "How would you test a vending machine?" or "Developer says It works on my machine"
- Negotiate defect severity vs release deadlines with engineering managers and product owners
- Build a portfolio with GitHub automation repositories and demonstrable defect logs

Lesson Modules:
[BEGINNER • 25 Minutes] Lesson 1: Mastering the STAR Framework for QA Behavioral Questions
Summary: Learn how to answer high-impact behavioral questions with real evidence and quantifiable achievements.
Key Detailed Topics:
- The STAR Framework breakdown: Situation, Task, Action (what YOU did), and Measurable Result
- Deconstructing: "What do you do if a developer rejects your bug saying it works on my machine?"
- Step-by-step resolution: clean cache, verify environment alignment, pair-debug, refer to Acceptance Criteria
- How to test physical/abstract objects in interviews (clarifying questions, functional, non-functional, boundary, stress)

' QA Interview Mastery Certification Quiz (1 Comprehensive Scenario, 75% Passing)
&¡ Daily Technical Interview Scenario & Live Rubric Evaluation

---

3. Workplace Defect Simulation Lab Scenarios

Students practice in a live sandbox e-commerce application (RootShop Checkout) with 4 real defects seeded for discovery and Jira bug logging:

• Defect: Cart Allows Zero or Negative Quantity [Severity: Critical]
Target Area: Shopping Cart Quantity Input
Technical Root Cause: Input fails to enforce min=1. Entering negative integers recalculates total to negative, generating fraudulent credits.

• Defect: Coupon SAVE50 Triggers Negative Balance [Severity: Critical]
Target Area: Discount Coupon Box
Technical Root Cause: Applying $50 discount on orders under $50 fails to clamp minimum total at $0.00, allowing zero/negative checkout.

• Defect: Phone Field Error Fails to Display Error Message [Severity: Major]
Target Area: Shipping Address Phone Number
Technical Root Cause: Form submission silently aborts on empty or invalid phone without focusing or presenting an accessible error message.

• Defect: Delivery Instructions Note Reflects Unescaped Text [Severity: Major]
Target Area: Delivery Notes Preview
Technical Root Cause: Courier preview card renders customer instructions without HTML sanitization, leaving system vulnerable to script injection.

4. Curriculum Revision & Customization Guide

To update or expand this curriculum, edit the primary data store at src/data/curriculumData.ts. You can add:
1. New Courses/Tracks: Cypress, Selenium Java, CI/CD with GitHub Actions, Performance testing with k6/JMeter.
2. More Lessons & Code Snippets: Gherkin syntax, Playwright fixtures, SQL window functions, JWT security tests.
3. Custom Certification Quizzes: Add scenario questions, multi-select questions, and pass thresholds.
4. Additional Defect Simulation Scenarios: API mocking bugs, slow network latency simulations, SQL injection flaws.